Position: Web Agent Research Needs Capture-Replay Infrastructure, Not More Handcrafted Replicas

Anonymous Authors[1]

Abstract

This is a position paper. We argue that web agent research should replace handcrafted website replicas with capture-replay infrastructure that turns real expert demonstrations into reusable, offline environments. Current benchmarks require heavy engineering yet cover a narrow, economically unrepresentative slice of web work, while proprietary environment providers concentrate access. Capture-replay offers a scalable alternative: a single expert session can produce a complete environment in minutes, grounded in real workflows and shareable via open-source tooling. We analyze the benchmark landscape and its cost structure, and present TRACE, a proof-of-concept pipeline that works on production sites including authenticated flows. We discuss limitations (reduced exploration, legal and ethical constraints, and replay determinism) and argue they are tractable through additional collection and community norms. We conclude with a call to action to redirect investment away from handcrafted replicas and toward capture-replay, and to document environment construction costs to enable fair methodological comparisons.

1. Introduction

> *"Web agents are hill climbing in the wrong direction."*

The past two years have witnessed remarkable progress in language-model agents for web and computer use. Systems built on frontier models can now navigate complex websites, fill forms, execute multi-step workflows, and retrieve information across diverse domains (Song et al., 2025; Wei et al., 2025; Murty et al., 2024; Maia et al., 2023; Zhang et al., 2023; Li et al., 2025; Zhou et al., 2023; Garg et al., 2025; Xie et al., 2024; Xu et al., 2024; He et al., 2024; Drouin et al., 2024). Benchmarks such as Mind2Web, WebArena, WebVoyager, WorkArena, REAL, OSWorld, BEARCUBS, BrowseComp, and TheAgentCompany have provided standardized evaluation environments that enable reproducible comparisons and have driven rapid capability improvements.

Yet beneath this progress lies a troubling structural problem: **the environments we use to train and evaluate web agents bear little resemblance to the tasks that would make these agents economically valuable.** The majority of benchmark tasks fall into categories we characterize as "deep research" (trivia-style multi-hop questions that rarely require browser automation), "information seeking" (simple lookups that could often be answered by a search engine), or "atomic execution" (short, isolated actions like clicking a button or filling a single field). Long-horizon, multi-step workflows that mirror real paid work (booking complex travel itineraries, configuring enterprise SaaS tools, executing financial transactions, managing e-commerce operations) remain dramatically underrepresented. More importantly, most benchmarks are not collected from people doing paid work; they are task templates designed for evaluability rather than traces of real economic activity.

This is not an oversight. It reflects a fundamental constraint: **handcrafted environments are extraordinarily expensive to build.**

Our position: The web agent research community should replace handcrafted replicas with capture-replay infrastructure. Rather than building website replicas, we should record experts completing tasks on live websites and transform those recordings into reusable, offline environments. A single expert demonstration can produce a complete environment in minutes, compared to the hours or days required to handcraft a replica. The tools exist; the decision is whether we continue paying the replica tax or move the field forward.

Consider the costs documented in recent benchmark papers:

TheAgentCompany's transparent reporting is particularly illuminating: building 175 realistic "employee-style" tasks

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

Table 1. Environment construction costs for prominent benchmarks.

Benchmark	Tasks	Reported Effort
WebArena	∼812	Months of development; self-hosting four functional website replicas
OSWorld	∼369	Hand-annotated tasks with initial state scripting and custom evaluators
TheAgentCompany	∼175	**3,000 person-hours** by 21 contributors over two months
REAL	∼112	Deterministic replicas of 11 real websites with custom harnesses

required 3,000 person-hours—the equivalent of 1.5 full-time engineers working for an entire year. The project involved 21 contributors including software engineers and project managers, with complex tasks requiring more than 10 hours each to design, implement, test, and verify. And these are among the most resource-rich academic teams in the field.

The implication is stark: at current construction costs, the academic community cannot build environments at the scale needed to cover the long tail of economically valuable browser tasks. We can produce hundreds of tasks, perhaps low thousands with extraordinary effort, but the space of valuable web workflows spans millions of distinct patterns across hundreds of thousands of websites.

These costs are not only financial. They consume scarce research time that could otherwise go toward agent design, failure analysis, and iteration. And because realistic environment construction is so expensive, evaluation suites remain small and often collapse to binary success metrics, which limits error analysis and makes it difficult to diagnose where, why, and how models fail.

This gap has not gone unnoticed by industry. A growing ecosystem of startups now builds browser environments for AI agent training (mec, 2025; pla, 2025; agi, 2025; kai, 2025). The business models vary: some host live environments, others deliver static datasets, and some offer both. What they share is a focus on creating replica websites and simulated workflows where AI agents can learn through reinforcement learning without triggering blocking mechanisms on real sites.

The scale of investment is remarkable. According to *The Information*, Anthropic has discussed spending more than $1 billion on RL environments over the next year (Zeff, 2025). Data-labeling giants like Scale AI, Surge, and Mercor are pivoting resources toward environment construction, with Surge reportedly generating $1.2 billion in revenue last year from AI lab contracts and recently spinning up a dedicated internal organization for RL environments. Mechanize, a startup focused exclusively on environments, offers software engineers $500,000 salaries to build them, far exceeding typical data-labeling contractor rates. Andreessen Horowitz general partner Jennifer Li summarized the landscape: "All the big AI labs are building RL environments in-house... but AI labs are also looking at third-party vendors that can create high-quality environments."

Leading AI labs treat these environments as core training infrastructure. **Browser environments have become a strategic asset: valuable, proprietary, and concentrated in the hands of well-funded organizations.**

The field is at an inflection point. Three trends are converging: (1) agents are becoming capable enough that evaluation on short, synthetic tasks no longer discriminates between systems, creating demand for long-horizon, realistic environments; (2) the cost of building such environments has created a two-tier ecosystem where frontier labs invest billions while academic groups work with hundreds of tasks; and (3) the resulting concentration raises safety concerns, as independent researchers cannot study systems trained on environments they cannot access. If the community does not change course, the gap will widen and become structural.

We believe this trajectory is problematic for the field. When the ability to train and iterate on realistic web agents depends on access to expensive proprietary infrastructure, the research community's capacity for independent investigation is constrained. Open science suffers. Reproducibility suffers. And the concentration of capability in a small number of actors raises broader concerns about the development trajectory of increasingly powerful autonomous systems. This issue affects not only academic groups, but also open-source LM companies operating on budgets that are tiny relative to frontier labs.

This paper argues for an alternative approach: capture-replay.

The core idea is simple: rather than handcrafting website replicas, we record an expert completing a task on a live website and transform that recording into a self-contained environment that can be replayed offline. A single expert demonstration (taking minutes to perform) produces a complete environment bundle including DOM states, network responses, screenshots, and interaction logs. Agents can then be evaluated (and potentially trained) against this frozen snapshot without contacting the live web.

Capture-replay is not a new concept in software engineering. Record-and-replay debugging, network mocking, and browser automation testing have used similar techniques for decades. But its application to web agent research has been surprisingly limited. We argue this represents a significant missed opportunity.

To demonstrate feasibility, we developed TRACE (Trajectory Recording and Capture Environments), an open-source

capture-replay pipeline. Building robust tooling required approximately 300 hours of engineering effort: analyzing hundreds of captured sessions, developing URL canonicalization heuristics, and iterating on replay matching. But once built, the tooling amortizes: we captured six diverse environments (GitHub, Amazon, Airbnb, Kayak, Uniqlo, Ultimate Guitar) in approximately 40 minutes of total collection time, including retries and quality checks. This is roughly 6–7 minutes per task versus the 10+ hours per task reported for handcrafted benchmarks like TheAgentCompany.

We emphasize "replace" deliberately. Handcrafted replicas consume scarce human capital but are not required to achieve determinism, controlled variation, or breadth. Those properties can be achieved by capture-replay at scale: multi-trajectory collection, record-on-miss expansion, and post-processing that enables controlled perturbations. The community's near-exclusive focus on replicas has slowed progress and narrowed evaluation; we argue the field should move past them.

The remainder of this paper develops this argument in detail:

- **Section 2** presents a taxonomy of existing benchmarks, revealing systematic gaps in economically valuable task coverage and analyzing the cost structure of current approaches.
- **Section 3** articulates the case for capture-replay, describing its potential advantages and presenting TRACE as a proof-of-concept implementation.
- **Section 4** addresses alternative views and counterarguments, including concerns about exploration latitude, legal and ethical considerations, and technical feasibility.
- **Section 5** presents a concrete call to action with specific recommendations for researchers, benchmark creators, and the broader community.

2. The Current Landscape: Expensive Replicas, Limited Coverage

2.1. A Taxonomy of Browser Agent Benchmarks

To understand the current state of web agent environments, we systematically analyzed eleven prominent benchmarks across three dimensions: **task category** (what kind of work the agent performs), **task horizon** (how many steps or how much time tasks typically require), and **economic grounding** (whether tasks resemble work someone would pay to have completed).

Several patterns emerge from this analysis:

Pattern 1: The effort-value tradeoff. Benchmarks that emphasize economically realistic tasks (TheAgentCompany, WorkArena, REAL) require dramatically more construction effort than those emphasizing information retrieval or synthetic reasoning (BrowseComp, GAIA). This is not coincidental. Realistic execution tasks require functional environments with state that actually changes, authentication flows that work, and evaluation criteria that verify real outcomes.

Pattern 2: Horizon compression. Even benchmarks labeled as "long-horizon" often consist of relatively short interaction sequences when measured in actual steps. Mind2Web 2 explicitly analyzed this phenomenon, finding that most existing benchmarks concentrate on short-horizon tasks (Li et al., 2025). Long-horizon, multi-session workflows that characterize real knowledge work remain rare.

Pattern 3: Horizon costs explode. As agents improve, the field is pushing toward longer-horizon evaluations (e.g., METR's work on measuring ability to complete long tasks (METR, 2025)). But handcrafting long-horizon tasks scales poorly: if some tasks already require 10+ hours to design and verify, the idea of 12-hour, multi-day, or week-long workflows is untenable. Replica construction cannot keep pace with the horizon lengths we need to measure.

Pattern 4: Domain concentration. Existing benchmarks heavily favor a small set of domains: e-commerce, content management, developer tools, and travel booking appear repeatedly, while vast categories of economically important web work (financial services, healthcare portals, government services, enterprise SaaS, professional services) remain largely uncovered.

Pattern 5: The live-web evaluation problem. Benchmarks that evaluate on live websites (Mind2Web, BrowseComp, BEARCUBS, WebVoyager) face continuous validity challenges as the web changes. Those that avoid this through replicas (WebArena, REAL) gain reproducibility but lose coverage and realism.

2.2. The Cost Structure of Environment Construction

Why are realistic browser environments so expensive to build? The costs compound across multiple dimensions:

Website replication costs. Creating a functional replica of even a moderately complex website requires reverse-engineering its interaction model, implementing sufficient backend logic to respond meaningfully to agent actions, and maintaining consistency as the original site evolves. REAL's approach (deterministic simulations of 11 real websites) required building custom evaluation harnesses that

Table 2. Taxonomy of existing web agent benchmarks. We observe a clear pattern: benchmarks with high economic grounding (TheAgentCompany, WorkArena, parts of REAL) require the most construction effort, while scalable benchmarks (Mind2Web, BrowseComp) tend toward tasks with limited economic value.

Benchmark	Category	Horizon	Econ. Grounding	Key Limitation
GAIA	Deep research	Medium	Low	General-assistant QA; many tasks not web-specific
BEARCUBS	Info seeking	Short–Medium	Low	QA-only; no state-changing actions; small (111 tasks)
BrowseComp	Deep research	Long	Low	QA-only; short answers; open-web drift
Mind2Web 2	Deep research	Long	Low	Agentic search QA; LLM-as-judge evaluation
Mind2Web	Info seeking / Exec	Short–Medium	Medium	Live-web trajectories; action-matching eval; no deterministic env
WebVoyager	Info seeking / Exec	Medium	Low–Medium	Live websites; GPT-4V-based eval; limited domain coverage
WebArena	Execution	Medium	Medium	Replica sites; limited domains; high construction cost
REAL	Execution	Short–Medium	Medium–High	Deterministic replicas; limited task count and domains
OSWorld	Execution	Short–Medium	Medium	Desktop-focused; heavy state setup and evaluators
WorkArena	Execution	Short–Medium	High	Single platform (ServiceNow); small (33 tasks); remote-hosted
TheAgentCompany	Execution	Medium	High	Realistic workflows but extraordinarily expensive to build

mix programmatic checks with LLM-based judgment (Garg et al., 2025).

State initialization costs. Realistic tasks often require specific preconditions: items in a shopping cart, emails in an inbox, projects configured in a particular way. OSWorld's task suite required extensive scripting to establish correct initial states for each of its 370 tasks (Xie et al., 2024). TheAgentCompany built an entire simulated company environment with interconnected services (Xu et al., 2024).

Evaluation complexity costs. When tasks have multiple valid solutions or require judgment about partial completion, evaluation itself becomes a substantial engineering challenge. The field has increasingly turned to LLM-as-a-judge approaches, but these introduce their own costs, biases, and reproducibility concerns (Xue et al., 2025).

Maintenance costs. The web changes constantly. Benchmarks evaluated on live sites require ongoing curation to verify task validity. Replica-based benchmarks must track upstream changes if they aim to remain representative.

Credential and access costs. Many economically valuable tasks require authenticated access to real services. Benchmarks either avoid these tasks, use synthetic accounts with limited functionality, or require evaluators to provide their own credentials. Each approach constrains coverage or reproducibility.

2.3. The Concentration of Environment Access

The expense of environment construction has predictable consequences for who can build and access high-quality environments.

As discussed above, a commercial ecosystem has emerged to fill this gap. These providers offer browser environments through various models: some host live replicas, others deliver packaged datasets, and some provide both. The environments enable something impossible on real websites: running thousands of AI agents simultaneously through trial-and-error learning without being blocked.

Leading AI labs use these commercial environments for training and evaluating their web agents. As frontier labs have exhausted most available text data for pretraining, reinforcement learning in simulated environments has become increasingly important. With per-task costs in the thousands to tens of thousands of dollars, these environments represent substantial investments, but ones that well-capitalized organizations can afford while academic groups generally cannot.

We also see this dynamic inside product companies building agents for their own products: teams are spun up to build replicas of their own UIs and workflows so they can train and evaluate at scale. This is yet another signal that handcrafted environments are expensive, bespoke infrastructure, not a

sustainable research path.

These training environments are typically proprietary and contract-bound, making it hard for open-source and academic research to access or reproduce the training conditions that drive frontier results.

The result is a two-tier system: well-funded labs train agents on high-quality, realistic environments that they cannot share, while the academic community and open-source LM companies work primarily with the limited set of open benchmarks. This concentration raises concerns about:

1. **Reproducibility:** If state-of-the-art results depend on proprietary training environments, the research community cannot fully verify or build upon them.
2. **Diversity:** Concentrated environment access may lead to agents that excel on specific task distributions while failing on the broader diversity of real-world needs.
3. **Safety:** Independent safety research requires access to the same environments used for capability development; concentration of access constrains this.

3. The Case for Capture-Replay

3.1. Core Concept

Capture-replay inverts the environment construction process. Rather than building a website replica and then defining tasks on it, we:

1. **Capture:** An expert performs a real task on a live website while an instrumented browser records everything: DOM states, network traffic, user interactions, screenshots, and video.
2. **Process:** A post-processing pipeline transforms raw captures into clean, reusable artifacts: high-level action sequences (a standardized tool-call DSL), extracted credentials (for secure handling), semantic checkpoints (for partial-credit evaluation), and filtered network logs (removing analytics and non-essential traffic).
3. **Replay:** A replay engine serves the captured assets locally, allowing agents to interact with a frozen snapshot of the original session without contacting the live web.

3.2. Advantage 1: Scalability

The most significant advantage of capture-replay is speed. **A single expert demonstration produces a complete environment in the time it takes to perform the task.**

Our proof-of-concept implementation (TRACE) captured six diverse tasks across GitHub, Amazon, Airbnb, Kayak, Uniqlo, and Ultimate Guitar in about 40 minutes of total collection time including retries (about 6:40 per task). Each capture produced hundreds of HTTP responses, dozens of DOM snapshots, and complete interaction logs—artifacts that would require days or weeks to handcraft.

This changes the economics of environment construction fundamentally. At capture-replay speeds:

- A single researcher could produce hundreds of environments per week
- Crowdsourced collection becomes feasible (we built a desktop app for non-technical collectors)
- The long tail of websites becomes accessible: any site an expert can use can become an environment

3.3. Advantage 2: Economic Grounding

Capture-replay environments are grounded in real tasks by construction. When an expert books an actual flight, configures a real SaaS tool, or completes a genuine e-commerce transaction, the resulting environment captures a workflow that someone demonstrably values enough to perform.

This contrasts with the synthetic task design required for replica-based benchmarks, where researchers must imagine what tasks would be valuable and then construct environments to support them. The imagination often falls short of reality: we design tasks we can evaluate rather than tasks that matter.

Capture-replay enables a different paradigm: **find people doing economically valuable work, record them, and build environments from their actual workflows.** This could connect web agent research more directly to real productivity gains.

3.4. Advantage 3: Democratization

Open-source capture-replay tooling can break the concentration of environment access. Any researcher with a browser can record environments from any website they can access. No proprietary infrastructure required, no licensing fees, no vendor lock-in.

This is not merely theoretical. TRACE is released as open-source software (URL redacted for double-blind review), and its captured environments are published on a public dataset host (URL redacted for double-blind review). The tooling is designed for extensibility: researchers can adapt it to their domains, contribute improvements, and build shared infrastructure without depending on commercial providers.

3.5. TRACE: A Proof of Concept

To demonstrate that capture-replay is technically feasible on modern production websites, we developed TRACE (Trajectory Recording and Capture Environments), an open-source pipeline implementing the full capture-process-replay workflow.

TRACE implements three core components: (1) a **collection** system using a stealth-configured Playwright browser that records DOM states, network traffic, user interactions, screenshots, and browser storage; (2) a **post-processing** pipeline that converts raw events to a standardized action DSL, extracts credentials for secure handling, identifies semantic checkpoints, and filters non-essential traffic; and (3) a **replay** engine that serves captured assets offline with deterministic request matching. Full implementation details are provided in Appendix A.

Demonstration dataset. We release six captured environments spanning GitHub, Amazon, Ultimate Guitar (authenticated flows with login), and Uniqlo, Kayak, Airbnb (unauthenticated). These cover e-commerce, travel search, social coding, and media sites, including sensitive interactions like payment flows. Statistics are provided in Appendix B.

The dataset is intentionally small: six tasks demonstrates *feasibility*, not comprehensive coverage. Building TRACE required approximately 300 hours of engineering effort, but once built, the six environments were captured in about 40 minutes total (∼6–7 minutes per task). This asymmetry (high tooling cost, low marginal collection cost) is precisely why capture-replay scales where replicas do not.

4. Alternative Views and Counterarguments

A position paper must engage seriously with opposing views. We address the most significant counterarguments to capture-replay:

4.1. "Capture-replay limits agent exploration"

The concern: Replica-based environments allow agents to explore freely: taking different paths, making mistakes, recovering from errors. Captured environments are constrained to the specific trajectory recorded; if an agent deviates, the replay may lack the network responses needed to continue.

Our response: This is not a fundamental limitation. The low cost of collection makes coverage of alternative paths a data problem, not an environment-construction problem. Two practical mechanisms address exploration directly:

1. **Multi-trajectory collection.** Capture multiple expert paths for the same task. These provide recovery behaviors and legitimate alternatives without any hand-built replica.
2. **Deterministic branching for exploration.** Replay can expose captured branches deterministically, allowing beam-search-style exploration over alternative paths. When an agent hits a missing branch, record-on-miss collection adds it and expands the environment.

In short, exploration constraints are solvable with more capture and deterministic branching, not a reason to keep handcrafted replicas.

4.2. "Legal and ethical concerns around captured content"

The concern: Capturing and redistributing website content raises legal questions (copyright, terms of service) and ethical questions (privacy, consent, potential for misuse).

Our response: These concerns are legitimate and require careful attention. We advocate for:

1. **Credential extraction and substitution.** TRACE's pipeline separates credentials from captured content, allowing environments to be shared without exposing real passwords or tokens.
2. **PII redaction.** Post-processing should identify and redact personally identifiable information beyond credentials.
3. **Restricted distribution.** Not all captured environments should be publicly released. Research-use agreements, access controls, and clear licensing can balance openness with responsibility.
4. **Ethical guidelines.** The community should develop shared norms around what captures are appropriate. We propose guidelines in Appendix C.

Importantly, replica-based benchmarks face analogous concerns that are often overlooked. WebArena clones the visual design and interaction patterns of Reddit, GitLab, and shopping sites. REAL builds "deterministic simulations" that reproduce real website behavior. These replicas copy copyrighted UI designs, reverse-engineer proprietary interaction models, and redistribute structural representations of commercial products. The legal status of benchmark environments is unsettled regardless of methodology. Capture-replay does not introduce novel legal risk categories; it inherits the same ambiguities that replica builders have navigated (often silently) for years. The difference is that capture-replay makes the provenance explicit, which may actually support clearer fair-use arguments for research purposes.

4.3. "LM-based matching introduces non-determinism"

The concern: TRACE uses LM-based disambiguation when multiple captured responses match an outgoing request. This introduces potential non-determinism across evaluation runs.

Our response: The LLM matcher is a proof-of-concept convenience, not a requirement. In principle, request matching should be fully deterministic for most websites by canonicalizing and matching on method, URL (scheme/host/path + normalized query), and request body parameters. Ambiguities can be resolved by strict tie-breakers (e.g., exact header matches, body hashes, or earliest-capture precedence) without model calls. A production system should default to deterministic matching and treat LLMs, if used at all, as a last-resort debugging tool.

4.4. "Six tasks doesn't prove scalability"

The concern: A proof-of-concept with six tasks doesn't demonstrate that capture-replay can scale to benchmark-sized collections or handle the diversity of the web.

Our response: Correct. Six tasks demonstrates technical feasibility, not scale. We claim only that capture-replay *can* work, not that we have built a complete benchmark.

However, we note:

1. **The time investment was minimal.** Six environments in about 40 minutes of total collection time (about 6:40 per task) suggests that hundreds are achievable with modest effort.

2. **Diversity was intentional.** We selected sites spanning e-commerce, social coding, travel, retail, and entertainment to demonstrate breadth.

3. **Challenging cases were included.** Authenticated flows, payment interactions, and dynamic content (map interfaces, infinite scroll) are represented.

4. **The small task count belies substantial engineering investment.** Building TRACE required approximately 300 hours of engineering effort: not to create six tasks, but to develop robust tooling that generalizes across sites. The six released tasks are the validation that this infrastructure works; the infrastructure itself is the contribution.

5. **Scaling is now an engineering challenge, not a research question.** Given working tooling, collecting more environments requires time and experts, not methodological breakthroughs.

We welcome the community to stress-test capture-replay at larger scales and report findings.

4.5. "Handcrafted replicas are necessary for controlled studies"

The concern: Certain research questions (robustness to perturbations, controlled ablations, systematic variation) require the control that only handcrafted environments provide.

Our response: We disagree. With sufficient capture breadth and tooling, capture-replay can support controlled studies without hand-built replicas:

1. **Multi-trajectory collections** provide alternative valid paths and recovery behaviors.

2. **Record-on-miss expansion** covers new branches by capturing additional sessions rather than engineering a replica.

3. **Post-processing and parameterization** can create controlled perturbations (DOM edits, resource swaps, replay-time filtering) while preserving fidelity to real websites.

4. **Deterministic replay** already provides the reproducibility needed for ablations.

We have not found a research question that strictly requires handcrafted replicas; the human-capital cost is not justified relative to capture-replay.

4.6. "If capture-replay works, why do commercial environment providers exist?"

The concern: Companies charge $3,000 to $50,000 per task for browser environments. These are sophisticated actors with access to the same techniques. If capture-replay were sufficient, why would labs pay these prices?

Our response: Commercial providers do use capture-replay methods. They are not building environments from scratch. Their value proposition is scale, customization, and handling the tedious engineering that most teams cannot or will not do.

Building robust capture-replay infrastructure is genuinely hard. TRACE required approximately 300 hours of engineering effort, involving pattern analysis across hundreds of websites, iterative refinement of URL canonicalization heuristics, debugging replay failures across diverse site architectures, and developing detection logic that generalizes beyond individual sites. This is tedious, unglamorous

work that does not produce publishable research artifacts—exactly the kind of work that commercial providers monetize.

The existence of commercial providers does not invalidate capture-replay; it validates that capture-replay is the underlying methodology. What providers sell is the engineering labor to make it work reliably at scale, plus customization layers (parameterized variations, API integrations, guaranteed coverage). Our argument is that this engineering should be done once and shared openly, not replicated behind paywalls at every organization. TRACE is a step toward that shared infrastructure.

4.7. "Training requires more exploration than evaluation"

The concern: TRACE is currently validated for evaluation, not RL training. Training via reinforcement learning requires exploration, trial-and-error, and feedback from mistakes. Captured environments cannot provide negative rewards for failed actions or support recovery from off-trajectory states.

Our response: TRACE is currently validated for evaluation, not RL training, but we see no fundamental barrier to training use, only engineering and data scale challenges. Notably, handcrafted replicas face the same challenge: supporting diverse exploration paths requires either engineering every possible interaction (expensive) or accepting limited coverage. The difference is that capture-replay can expand coverage in minutes per trajectory, while replicas require significant engineering effort per new behavior. We note that while details remain proprietary, commercial environment providers appear to use capture-based methods (not pure handcrafted replicas) to achieve faster environment generation, suggesting the industry has already moved in this direction.

Consider what multi-trajectory capture-replay provides:

1. **A graph of valid transitions, not a single path.** With sufficient trajectory diversity, captured environments form a state-action graph suitable for offline RL and batch RL methods.
2. **Graded reward signals via checkpoints.** TRACE's checkpoint system enables partial-credit feedback even when agents fail the final goal.
3. **Imitation learning and behavioral cloning.** Captured expert demonstrations are directly usable for learning from demonstration.
4. **Record-on-miss expansion.** When agents hit uncaptured branches, the missing transitions can be recorded and added.

The key question is data scale: how many trajectories are needed for effective RL? This is empirical. We also note that **evaluation is the more urgent bottleneck**—most academic groups cannot evaluate on realistic long-horizon tasks because the environments do not exist.

4.8. "The modern web actively resists being captured"

The concern: Websites deploy sophisticated anti-automation measures: CAPTCHA challenges, bot detection, device fingerprinting, behavioral analysis, and rate limiting.

Our response: This is true, and one reason TRACE required substantial engineering effort. We found that:

1. **Many high-value sites are capturable** with sufficient engineering. Stealth browser configuration and realistic interaction timing enabled successful capture across all six demonstration sites.
2. **Anti-bot measures vary widely in sophistication.** The landscape is heterogeneous, not uniformly hostile.
3. **Replay is easier than capture.** Once traffic is captured, replay serves responses locally without triggering anti-bot measures.

We view anti-bot resistance as a practical obstacle that increases engineering cost, not a fundamental barrier. The question is whether the investment is lower than handcrafting replicas. We believe it is, especially when tooling is shared.

4.9. "Frozen captures do not generalize to live sites"

The concern: A model evaluated on captures from January 2025 may fail on the same site in March 2025 when UI layouts change.

Our response: This is a genuine limitation. Capture-replay trades temporal validity for reproducibility and cost efficiency. We offer three observations:

1. **Replica-based benchmarks face the same problem.** WebArena's Reddit clone does not track Reddit's actual UI evolution.
2. **Reproducibility has independent value.** Even if capture-to-live transfer is imperfect, comparing agents on identical environments enables scientific progress.
3. **Periodic recapture can track drift.** Unlike replicas requiring substantial rework, capture-replay environments can be refreshed by recording new sessions.

We call for empirical research measuring capture-to-live transfer.

5. Call to Action

We conclude with specific recommendations:

5.1. For Research Groups and Open-Source LM Companies

1. **Redirect resources away from replica construction.** Stop building new handcrafted replicas and invest in capture-replay tooling and collection capacity.
2. **Collect domain-specific environments.** Research groups with domain expertise (finance, healthcare, enterprise software) can capture environments difficult for generalist teams.
3. **Publish captured environments with safeguards.** Share environments under research-use licenses with credential substitution and PII redaction. Document capture methodology for reproducibility.
4. **Expand capture breadth.** Use record-on-miss, multi-trajectory collection, and replay-time perturbations to expand exploration latitude without handcrafted replicas.

5.2. For Benchmark Creators

1. **Document environment construction costs.** Transparent reporting of person-hours, like TheAgentCompany's exemplary disclosure, enables informed methodology comparisons. We recommend all benchmark papers include this information.
2. **Adopt capture-replay for evaluation suites.** Replace new replica-based benchmarks with capture-replay collections to scale coverage and enable deeper error analysis.
3. **Plan migrations from replicas.** For existing replica-based suites, publish a capture-replay transition plan and a timeline for deprecating handcrafted environments.
4. **Establish shared infrastructure.** Coordinate on common tooling, formats, and distribution mechanisms for captured environments to reduce duplication and enable interoperability.

5.3. For the Broader Community

1. **Adopt and refine ethical guidelines for environment capture.** We propose comprehensive guidelines in Appendix C addressing consent frameworks, legal considerations, privacy protections, security risks, and domain-specific requirements. Community discussion should refine and formalize these norms.
2. **Create registries of captured environments.** Centralized, searchable collections would reduce duplication and enable broader access. Hugging Face and similar platforms could host these.
3. **Pressure commercial providers toward openness.** Encourage environment platforms to release subsets of their collections for research use, or to adopt open standards that reduce lock-in.
4. **Investigate legal frameworks.** Clearer understanding of copyright, ToS, and data protection implications would benefit the entire field. Legal scholarship on AI training data has relevant precedents.

5.4. Concrete Next Steps

For those convinced by our argument, we suggest immediate actions:

1. **Try TRACE.** Clone the repository, capture an environment from a website you use, replay it offline. Experience the workflow firsthand.
2. **Identify valuable tasks in your domain.** What browser workflows would genuinely save time or money if automated? These are candidates for capture.
3. **Contribute to shared infrastructure.** Submit captured environments to public registries. Report issues with capture tooling. Propose improvements to post-processing pipelines.
4. **Advocate within your organization.** If you work at a lab with access to commercial environments, advocate for releasing research subsets or contributing to open alternatives.

6. Conclusion

The web agent research community has made real progress, but handcrafted replicas are now the bottleneck. They are slow and expensive to build, lock us into small, synthetic task sets, and collapse evaluation to binary success. As task horizons stretch to hours and days, replica construction becomes untenable, and the ecosystem splits into a two-tier system where only well-funded labs and product companies can afford realistic environments.

Capture-replay replaces that bottleneck with fast, economically grounded environments collected from real work. Deterministic replay, multi-trajectory collection, and branching

give us coverage and analysis depth without hand-built replicas. The remaining challenges are about collection logistics and responsible sharing, not environment engineering.

Our position, restated: The web agent research community should replace handcrafted replicas with capture-replay infrastructure. Stop building new replicas, invest in capture tooling and large-scale collection, and standardize sharing so long-horizon, economically valuable tasks become accessible to everyone. The tools exist; the decision is whether we continue paying the replica tax or move the field forward.

References

AGI Inc. https://agi.inc, 2025.

Kaizen. https://www.kaizenautomation.com, 2025.

Mechanize. https://www.mechanize.work, 2025.

Plato. https://plato.so, 2025.

Drouin, A. et al. WorkArena: How capable are web agents at solving common knowledge work tasks? *arXiv preprint arXiv:2403.07718*, 2024.

Garg, D. et al. REAL: Benchmarking autonomous agents on deterministic simulations of real websites. *arXiv preprint arXiv:2504.11543*, 2025.

He, H. et al. WebVoyager: Building an end-to-end web agent with large multimodal models. *arXiv preprint arXiv:2401.13919*, 2024.

Li, Z. et al. Mind2Web 2: Evaluating agentic search with agent-as-a-judge. *arXiv preprint arXiv:2506.21506*, 2025.

Maia, M. J. A. et al. GAIA: A benchmark for general AI assistants in the wild. *arXiv preprint arXiv:2311.12983*, 2023.

METR. Measuring AI ability to complete long tasks. https://metr.org/blog/2025-03-19-measuring-ai-ability-to-complete-long-tasks/, 2025.

Murty, S. et al. NNetNav: Unsupervised learning of browser agents through environment interaction in the wild. *arXiv preprint arXiv:2410.02907*, 2024.

Song, Y. et al. BEARCUBS: A benchmark for computer-using web agents. *arXiv preprint arXiv:2503.07919*, 2025.

Wei, J. et al. BrowseComp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*, 2025.

Xie, T. et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972*, 2024.

Xu, F. F. et al. TheAgentCompany: Benchmarking LLM agents on consequential real world tasks. *arXiv preprint arXiv:2412.14161*, 2024.

Xue, T. et al. An illusion of progress? assessing the current state of web agents. *arXiv preprint arXiv:2504.01382*, 2025.

Zeff, M. Silicon valley bets big on 'environments' to train AI agents. *TechCrunch*, September 2025. https://techcrunch.com/2025/09/21/silicon-valley-bets-big-on-environments-to-train-

Zhang, C. et al. Mind2Web: Toward a generalist agent for the web. *arXiv preprint arXiv:2306.06070*, 2023.

Zhou, S. et al. WebArena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.

A. TRACE Implementation Details

This appendix provides technical details on the TRACE implementation. The complete source code is available at a URL redacted for double-blind review.

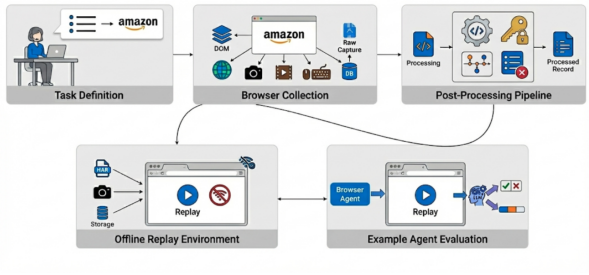

Figure 1. TRACE pipeline: collection, post-processing, offline replay, and agent evaluation.

A.1. Collection Architecture

TRACE's collection system is built on Playwright with a stealth-configured Chromium browser designed to avoid anti-automation detection while maintaining full recording fidelity.

Browser Configuration. The collector launches Chromium with carefully selected arguments to evade bot detection:

```
--disable-blink-features=
    AutomationControlled
--disable-features=VizDisplayCompositor
--no-sandbox
--disable-web-security
--enable-automation=false
```

The context is configured with realistic defaults: 1366×768 viewport, `en-US` locale, `America/New_York` timezone, and standard HTTP headers.

Event Recording. The `Recorder` class captures events across multiple categories:

Table 3. Event categories captured by TRACE.

Category	Event Types	Trigger
State:Page	load, domcontentloaded	Page lifecycle
State:Browser	navigated, back	Navigation
Action:User	click, input, submit	User interactions

For each triggering event, the recorder captures: (1) an accessibility snapshot (YAML-formatted tree of up to 400 interactive elements), (2) screenshots via Chrome DevTools Protocol, (3) full HAR network recording, and (4) browser state (cookies, localStorage, sessionStorage).

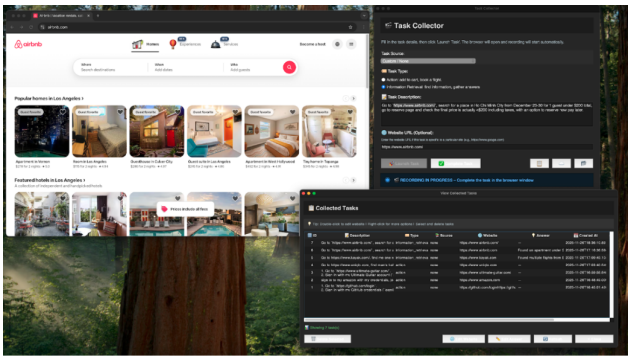

Figure 2. Collecting a task using the Tkinter desktop app.

Data Storage. Each collection produces: SQLite database with task metadata, per-step DOM snapshots, screenshots, video recordings, and a capture bundle containing manifest, HAR file, storage state, and cached resources.

A.2. Post-Processing Pipeline

The post-processing pipeline transforms raw captures into clean artifacts through four stages:

Stage 1: Tool-Call Parsing. Raw browser events are converted to a standardized DSL (`go_to`, `click`, `type`, `scroll`) with element information and navigation consequences.

Stage 2: Credential Extraction. An LLM-based extractor identifies credential-related interactions by analyzing tool calls and DOM context. Extracted credentials are stored separately, enabling credential substitution during sharing.

Stage 3: Checkpoint Selection. An LLM analyzes the full trajectory to identify 2–3 semantically meaningful intermediate states suitable for partial-credit evaluation.

Stage 4: Ignore-List Construction. Network traffic is analyzed to identify hosts and URL patterns that can be safely omitted (analytics, ad networks, non-essential third-party services). This stage discards approximately 50% of captured HTTP requests.

A.3. Replay Engine

The replay module reconstructs captured sessions through multi-stage request matching:

Request Routing. When the browser issues a request during replay: (1) URLs matching ignore patterns are aborted; (2) exact matches on method and URL base are used directly; (3) character-frequency similarity scores handle dynamic query parameters; (4) LLM disambiguation resolves remaining ambiguities.

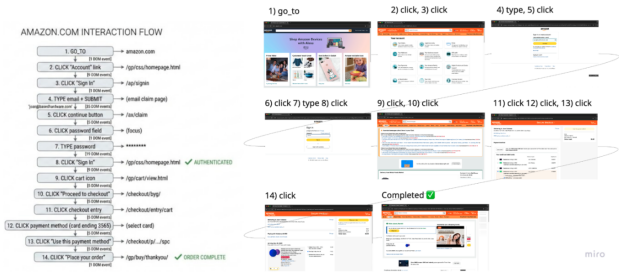

Figure 3. Offline replay and tool-call parsing for an Amazon task.

Response Fulfillment. Once a HAR entry is selected, response headers are extracted, bodies decoded, and the request fulfilled via Playwright's route API.

Caching. LLM match decisions are cached for determinism. The `--ignore-cache` flag forces fresh matching for debugging.

A.4. Evaluation Harness

TRACE includes a minimal evaluation runner: (1) load capture bundle and configure HAR replay routing; (2) initialize agent with task description; (3) compare agent trajectory against human checkpoints using semantic similarity; (4) assess binary success against expected outcome.

B. Captured Environment Statistics

Column definitions:

- **Clicks:** Number of click events recorded
- **Tools:** Number of high-level tool calls after post-processing
- **Creds:** Whether the task involved credential entry (login flows)
- **HTTP:** Total HTTP request/response pairs in HAR file (before ignore-list filtering)
- **Cache:** Number of cached static resources
- **Shots:** Number of screenshots captured
- **DOM:** Number of DOM/accessibility snapshots captured
- **Time(s):** Task execution duration in seconds (successful run only)
- **Size:** Total capture bundle size on disk

Aggregate totals: Total HTTP requests: 7,725 (before filtering; ∼50% discarded by ignore-list); Total cached resources: 3,092; Total screenshots: 120; Total DOM snapshots: 160; Total storage: 663 MB.

Note on collection time: The Time(s) column reflects the duration of a successful task execution. Actual collection effort is approximately 6:40 per task when accounting for retries due to collection errors, website issues, or mistakes during demonstration. The six-task dataset required about 40 minutes of total collection time.

B.1. Task Descriptions

Task 1: GitHub (Authenticated). Sign in, star a repository, search for and follow a user. Exercises authentication, navigation, and account actions.

Task 2: Amazon (Authenticated). Sign in, navigate to deals, find discounted item, add to cart, proceed toward checkout. Exercises e-commerce workflow including authentication, filtering, and cart management.

Task 3: Ultimate Guitar (Authenticated). Sign in, search for tabs, interact with playback controls. Exercises authentication on media-heavy site.

Task 4: Uniqlo (Unauthenticated). Browse catalog, apply filters, view product details, add to cart. Exercises e-commerce browsing without authentication.

Task 5: Kayak (Information Retrieval). Search flights with constraints, apply filters, compare results. Exercises travel search with complex filtering.

Task 6: Airbnb (Information Retrieval). Search accommodations with filters, browse listings with map interaction. Exercises map-based search with filtering.

B.2. Observations

Several patterns emerge from the collected data:

1. **HTTP traffic scales with site complexity.** Ultimate Guitar generated the most HTTP requests (2,752), likely due to media-heavy content and third-party integrations. Travel sites (Kayak, Airbnb) had moderate traffic despite complex UIs.
2. **Bundle size correlates with cached resources.** Uniqlo and Kayak, despite different task types, both required substantial cached resources (783 and 452 respectively), resulting in larger bundle sizes.
3. **Authenticated tasks were faster.** Tasks 1–3 (authenticated) averaged 54.4 seconds, while tasks 5–6 (information retrieval, unauthenticated) averaged 107.3

Table 4. Complete statistics for the six captured demonstration environments.

Task	Website	Type	Clicks	Tools	Creds	HTTP	Cache	Shots	DOM	Time(s)	Size
1	github.com	Action	11	8	Yes	715	311	12	35	39.0	64 MB
2	amazon.com	Action	14	11	Yes	1,199	468	20	31	69.2	94 MB
3	ultimate-guitar.com	Action	19	15	Yes	2,752	534	20	46	54.9	117 MB
4	uniqlo.com	Action	11	10	No	1,315	783	12	12	46.1	128 MB
5	kayak.com	Info	14	11	No	816	452	32	17	110.7	154 MB
6	airbnb.com	Info	15	12	No	928	544	24	19	103.9	106 MB

seconds. This likely reflects the more exploratory nature of information retrieval tasks.

4. **DOM snapshot frequency varies by interaction pattern.** Ultimate Guitar (46 snapshots) and GitHub (35 snapshots) had higher snapshot counts due to more frequent triggering events (clicks, form submissions).

Even accounting for retries and mistakes, total collection effort for all six environments was approximately 40 minutes ($\sim$6:40 per task), demonstrating the efficiency of capture-replay compared to handcrafted environment construction which can require hours or days per task.

C. Ethical and Safety Guidelines

This appendix proposes guidelines for responsible capture and distribution of browser environments.

C.1. Legal Considerations

Copyright. Captured environments contain copyrighted material. Fair use doctrines may permit research use, but this is untested for AI training. We recommend documenting fair use rationale and limiting distribution to research contexts. Notably, replica-based benchmarks face identical concerns: WebArena reproduces Reddit's interface; REAL simulates commercial sites.

Terms of service. Most websites prohibit automated access in ToS, though enforceability varies (*hiQ v. LinkedIn*). We recommend avoiding high-volume systematic capture and respecting robots.txt.

CFAA considerations. Post-*Van Buren* (2021), mere ToS violations are not CFAA crimes, but circumventing access controls may be. Capture only from accounts you own or have authorization to use.

C.2. Privacy and Data Protection

Captures contain personal data requiring attention under GDPR/CCPA:

1. **Informed consent.** Demonstrators should consent to capture and distribution.
2. **PII detection and redaction.** Post-processing should detect and redact names, emails, addresses, financial data, and government identifiers.
3. **Third-party minimization.** Avoid capturing pages with extensive third-party PII unless necessary.
4. **Credential extraction.** TRACE separates authentication data from content; extend this to general PII.

C.3. Security Risks

Credential exposure. Captures may contain passwords, session cookies, API keys, and tokens. Mitigations: mandatory credential extraction, secure storage, credential rotation after capture, and preference for short-lived tokens.

Vulnerability disclosure. Captures may reveal security flaws. Review captures before distribution; establish responsible disclosure procedures; withhold captures revealing serious vulnerabilities.

C.4. Misuse Potential

Capture-replay has dual-use risks. Environments could train agents for automated fraud, credential stuffing, phishing, or surveillance.

Mitigations: Access controls for sensitive domains (finance, healthcare, government); licensing that prohibits malicious applications; consider whether certain environment types (payment completion, account creation) should be captured at all; tiered access balances openness with responsibility.

Our view: Proprietary concentration does not eliminate misuse risk; it prevents independent safety research. Transparent infrastructure enables collective norm development.

C.5. Three-Tier Consent Framework

C.6. Domain-Specific Notes

- **Healthcare:** Never capture real patient data; treat as Tier 3

Table 5. Proposed consent framework for captured environments.

Tier	Scope	Requirements	Distribution
1: Self	Own accounts	Consent, cred extraction	Public after processing
2: Auth	Others' demos	IRB, written consent, PII redaction	Research license
3: Sensitive	Healthcare, finance, govt	Domain compliance, legal review	Institutional agreements

- **Financial:** Stop before payment completion; never capture real card data
- **Government:** Jurisdiction-specific regulations apply
- **Enterprise SaaS:** Obtain written authorization from system owners

C.7. Community Infrastructure

We call for: shared registries with access controls (e.g., Hugging Face), open-source compliance tooling (PII detection, credential extraction), community review for sensitive domains, and incident response procedures.

C.8. Limitations

These guidelines are not legal advice, cannot anticipate all risks, and depend on community adoption. Strict controls may recreate capability concentration. Despite limitations, explicit guidelines are preferable to the implicit norms around replicas, which sidestep these questions entirely.